

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5117

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

*I Harshini R N (192421175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Harshini R N

Annexure A
Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

1. IPSec Authentication Header (AH)

Aim:

To simulate IPSec AH and generate and verify a MAC for IP packet integrity.

Algorithm:

1. Read the IP packet and secret key.
2. Generate HMAC using SHA-256.
3. Attach the MAC to the packet.
4. For verification, generate HMAC again using the same key.
5. Compare both MAC values.
6. Display whether the packet is authentic or modified.

Python Program:

```
import hmac
import hashlib

def generate_mac(packet, key):
    return hmac.new(key.encode(), packet.encode(), hashlib.sha256).hexdigest()

packet = input("Enter IP packet: ")
key = input("Enter secret key: ")

mac = generate_mac(packet, key)
print("Generated AH MAC:", mac)

received_packet = input("Enter received IP packet: ")
received_mac = generate_mac(received_packet, key)

if hmac.compare_digest(mac, received_mac):
    print("AH Verification: Packet is authentic and intact.")
else:
    print("AH Verification: Packet is modified or invalid.")
```

Sample Input:

```
Enter IP packet: HELLO_IP_PACKET
Enter secret key: mysecret
Enter received IP packet: HELLO_IP_PACKET
```

Sample Output:

```
Generated AH MAC: 8f7...
AH Verification: Packet is authentic and intact.
```

2. IPSec Encapsulating Security Payload (ESP)

Aim:

To implement ESP for encryption and decryption of an IP packet payload.

Algorithm:

1. Read the plaintext payload and secret key.
2. Generate a keystream from the key.
3. XOR the payload with the keystream to encrypt it.
4. XOR the ciphertext with the same keystream to decrypt it.
5. Display the encrypted and decrypted payload.

Python Program:

```
import hashlib

def keystream(key, length):
    stream = b""
    counter = 0

    while len(stream) < length:
        stream += hashlib.sha256(
            (key + str(counter)).encode()
        ).digest()
        counter += 1

    return stream[:length]

def encrypt(data, key):
    data = data.encode()
    stream = keystream(key, len(data))
    return bytes(a ^ b for a, b in zip(data, stream))

def decrypt(data, key):
    stream = keystream(key, len(data))
    return bytes(a ^ b for a, b in zip(data, stream))

payload = input("Enter IP packet payload: ")
key = input("Enter secret key: ")

ciphertext = encrypt(payload, key)

print("Encrypted payload:", ciphertext.hex())

plaintext = decrypt(ciphertext, key)
print("Decrypted payload:", plaintext.decode())
```

Sample Input:

```
Enter IP packet payload: Secure IP Payload
Enter secret key: espkey
```

Sample Output:

```
Encrypted payload: 7a91...
Decrypted payload: Secure IP Payload
```

3. IPSec Security Association (SA) Management

Aim:

To create, search, and delete Security Associations in IPSec.

Algorithm:

1. Create an SA database using a dictionary.
2. Store SA details using an SPI as the identifier.
3. Search for an SA using its SPI.
4. Delete the SA when required.
5. Display the current SA database.

Python Program:

```
sa_database = {}

def create_sa(spi, src, dst, algorithm):
    sa_database[spi] = {
        "Source": src,
        "Destination": dst,
        "Algorithm": algorithm
    }
    print("Security Association created.")

def lookup_sa(spi):
    if spi in sa_database:
        print("SA Found:", sa_database[spi])
    else:
        print("SA not found.")

def delete_sa(spi):
    if spi in sa_database:
        del sa_database[spi]
        print("Security Association deleted.")
    else:
        print("SA not found.")
```

```
spi = input("Enter SPI: ")
src = input("Enter source IP: ")
dst = input("Enter destination IP: ")
algorithm = input("Enter security algorithm: ")

create_sa(spi, src, dst, algorithm)

print("\nSA Lookup:")
lookup_sa(spi)

print("\nSA Deletion:")
delete_sa(spi)

print("\nSA Database:", sa_database)
```

Sample Input:

```
Enter SPI: 1001
Enter source IP: 192.168.1.10
Enter destination IP: 192.168.1.20
Enter security algorithm: AES
```

Sample Output:

Security Association created.

SA Lookup:

```
SA Found: {'Source': '192.168.1.10', 'Destination': '192.168.1.20',
'Algorithm': 'AES'}
```

SA Deletion:

Security Association deleted.

SA Database: {}

4. Simplified PGP

Aim:

To implement a simplified PGP system using public-key encryption and digital signatures.

Algorithm:

1. Generate an RSA key pair.
2. Create a digital signature by hashing the message and signing the hash with the sender's private key.
3. Encrypt the message using the receiver's public key.
4. Receiver decrypts the message using the private key.

5. Verify the digital signature using the sender's public key.
6. Display the verification result.

Python Program:

```
import hashlib

# Simplified public/private key demonstration
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17
d = 2753

message = input("Enter email message: ")

# Digital signature
digest = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
signature = pow(digest, d, n)

# Simplified encryption
encrypted = [pow(ord(ch), e, n) for ch in message]

print("Encrypted message:", encrypted)
print("Digital signature:", signature)

# Decryption
decrypted = ''.join(chr(pow(x, d, n)) for x in encrypted)

# Signature verification
received_digest = int(
    hashlib.sha256(decrypted.encode()).hexdigest(), 16
) % n

verified_digest = pow(signature, e, n)

print("Decrypted message:", decrypted)

if received_digest == verified_digest:
    print("Digital Signature: VALID")
else:
    print("Digital Signature: INVALID")
```

Sample Input:

Enter email message: Meet me at 10 AM

Sample Output:

Encrypted message: [...]

Digital signature: 1234
Decrypted message: Meet me at 10 AM
Digital Signature: VALID

5. Email Spam Classification

Aim:

To classify emails as spam or legitimate using header and content-based filtering.

Algorithm:

1. Read the sender, subject, and email content.
2. Define common spam keywords.
3. Check the sender and message for suspicious patterns.
4. Increase the spam score when suspicious words are found.
5. Compare the score with a threshold.
6. Classify the email as Spam or Legitimate.

Python Program:

```
spam_words = [  
    "win", "winner", "free", "prize", "offer",  
    "urgent", "click", "money", "lottery", "congratulations"  
]  
  
sender = input("Enter sender email: ")  
subject = input("Enter email subject: ")  
content = input("Enter email content: ")  
  
text = (sender + " " + subject + " " + content).lower()  
  
score = 0  
  
for word in spam_words:  
    if word in text:  
        score += 1  
  
if "@" not in sender:  
    score += 2  
  
print("Spam Score:", score)  
  
if score >= 2:  
    print("Classification: SPAM")  
else:  
    print("Classification: LEGITIMATE")
```


Sample Input:

Enter sender email: unknown@example.com

Enter email subject: Congratulations! You won a free prize

Enter email content: Click here to claim your free money.

Sample Output:

Spam Score: 5

Classification: SPAM